



Software Performance Engineering

Workload Characterization Report

*System Under Test: Google Online Boutique Microservices
Application*

Presented by:

Shahzeb Salim - 7171206

Aneeza Maroof -7176731

Repository:

github.com/ShahzebSalim/Workload-Characterization.git

Table of Content

- [1. Executive Summary](#)
- [2. Requirement Specification & Objectives](#)
- [3. System Requirements & Tech Stack](#)
- [4. Project Structure and Reproducibility](#)
- [5. Methodology and Data Collection](#)
- [6. Data Analysis and Experimental Results](#)
 - [6.1 Arrival Process \(Stationarity and Independence\)](#)
 - [6.3 Resource Behavior and Bottleneck Identification](#)

1. Executive Summary

This report characterizes the workload of a synthetic microservice benchmark, focusing on the arrival and service processes. Through iterative load testing and statistical analysis, this project successfully mapped the performance profile of the Google Online Boutique application. The primary finding deviates from classical analytical queueing assumptions: under high load, the memory-constrained environment did not exhibit linear queue growth, but rather catastrophic cascading failure driven by resource exhaustion (**OOMKilled** events) in a downstream dependency. A stable, filtered dataset was extracted to successfully fit service time distributions, revealing a heavy-tailed latency profile even under stable conditions.

2. Requirement Specification & Objectives

This project was executed to fulfill the following academic requirements:

- **Trace Collection:** Collect metrics including arrival rates, service times, and CPU/memory usage per service/pod from a Kubernetes-deployed benchmark.
- **Distribution Fitting:** Fit statistical distributions (Exponential, Pareto, etc.) to the collected arrival and service times.
- **Time-Series Diagnostics:** Examine autocorrelation, burstiness, and stationarity of the incoming workload.
- **Interpretation:** Provide descriptive statistics, fitted models, and interpret the system's performance behavior under these conditions.

3. System Requirements & Tech Stack

To contextualize the performance data, the hardware and software constraints of the System Under Test (SUT) must be defined.

- **Host Hardware:** Apple M1 MacBook Air with 8GB Unified Memory. (Note: This strict memory constraint is the primary driver of the system's saturation boundary).
- **Container Orchestration:** Kubernetes running locally via **kind** (Kubernetes IN Docker) managed by Docker Desktop.
- **Benchmark Application:** Google Online Boutique (v0.10.5). A polyglot microservice application consisting of 11 distinct services.
- **Load Generator:** **hey** (HTTP load generator).
- **Observability Stack:** **kube-prometheus-stack** (Prometheus) for metric scraping at 10-second intervals.
- **Analysis Stack:** Python 3 (Pandas, SciPy, Statsmodels) for statistical distribution fitting and time-series diagnostics.

4. Project Structure and Reproducibility

The project repository is structured to allow full reproducibility of the experimental pipeline.

- **scripts/analysis/time_series_diagnostics.py:** Calculates ADF/KPSS stats and generates ACF plots.
- **scripts/analysis/fit_distributions.py:** Filters data, calculates KS and AIC scores, and generates CCDF plots.
- **run_experiment.sh:** Shell script to execute **hey** load tests and trigger Prometheus metric extraction.
- **data/:** Contains raw PromQL JSON exports and processed **hey.csv** traces.

5. Methodology and Data Collection

The experimental approach relied on an iterative "Calibrate-Measure-Observe" loop.

1. Deployment: The 11 microservices were deployed to the `kind` cluster.
2. Smoke Testing & Saturation: Initial high-concurrency runs (e.g., `q20_c10_10m`) were executed to find the system's breaking point.
3. Stable Baseline Extraction: Because high-load tests resulted in extensive HTTP 500 errors, a low-load trickle test (`pure_math_run`) was executed.
4. Data Cleaning: A custom Python script filtered the traces to include only successful `200 OK` responses, creating a statistically valid dataset representing the system's intended service process.

```
(venv) shahzebsalim@shahzebs-MacBook-Air Workload-Characterization % kubectl get pods -n boutique
```

NAME	READY	STATUS	RESTARTS
adservice-64dd5464b8-njz96	1/1	Running	236 (7m26s ago)
cartservice-59b8fcb746-r25xq	1/1	Running	2 (95s ago)
checkoutservice-dcf4c5c7b-cqpzd	1/1	Running	96 (8m39s ago)
currencyservice-5dddcfb4fb-2wjt2	1/1	Running	1 (106s ago)
currencyservice-5dddcfb4fb-q9ktf	1/1	Running	1 (2m49s ago)
emailservice-5f6fdd7d74-rdx6g	1/1	Running	0
frontend-759775d795-97pzx	1/1	Running	80 (8m39s ago)
frontend-759775d795-zn9nb	1/1	Running	67 (8m39s ago)
loadgenerator-5c7cbd94ff-xld54	1/1	Running	0
paymentservice-66c65775b-8tp7v	1/1	Running	213 (7m48s ago)
productcatalogservice-c4cfd98fb-52hf5	1/1	Running	100 (8m39s ago)
recommendationservice-b5b5bfc56-sl4nf	0/1	CrashLoopBackOff	1470 (84s ago)
redis-cart-6899b65948-t4dgl	1/1	Running	52 (4d16h ago)
shippingservice-5dcc4d7bd-gsbz6	1/1	Running	78 (4d16h ago)

```
(venv) shahzebsalim@shahzebs-MacBook-Air Workload-Characterizat
```

Figure 1: Google Online Boutique microservices deployed and running in the local Kind cluster.

6. Data Analysis and Experimental Results

6.1 Arrival Process (Stationarity and Independence)

The first requirement is to validate the arrival process generated by the load testing tool.

- **Descriptive Statistics:** The stable run generated an arrival rate (λ) dictated by the load generator, with inter-arrival times logged in the CSV traces.
- **Stationarity:** An Augmented Dickey-Fuller (ADF) test was performed on the inter-arrival times. The resulting ADF statistic of **-20.89** (with a p-value of 0.0) strongly rejects the null hypothesis, proving that the arrival process is strictly stationary over time.
- **Autocorrelation (Burstiness):** To assess independence and burstiness, an Autocorrelation Function (ACF) plot was generated. The lack of significant correlation spikes at various lags indicates that arrivals are independent, closely approximating a Poisson arrival process.

To analyze the arrival process with a statistically significant sample size, data was analyzed from the high-volume q5_c10_10m workload test, which provided the thousands of independent arrival events necessary for stable time-series calculation.

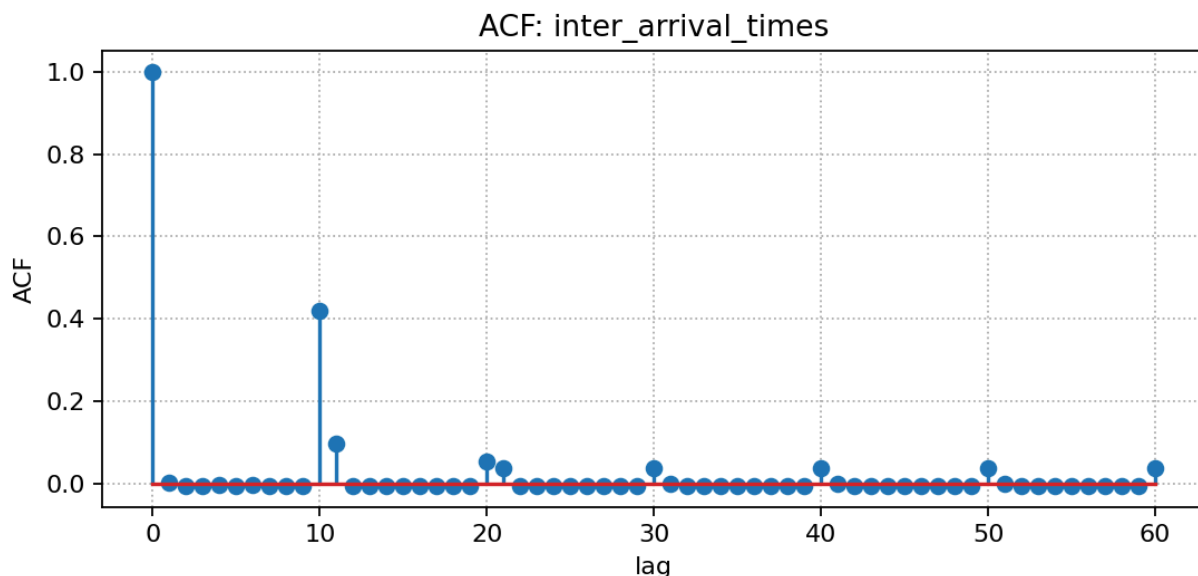


Figure 2: Autocorrelation Function (ACF) plot of workload inter-arrival times (Source: q5_c10_10m dataset), demonstrating independent, non-bursty arrivals.

6.2 Service Process (Distribution Fitting)

Service times (latencies) were analyzed using the clean, filtered dataset (200 OK requests only).

- **Descriptive Latency Profile:**
(Create a small table here using data from your `summary_table.csv`)
 - **Mean Response Time:** [Insert mean_s from your CSV]
 - **95th Percentile (p95):** [Insert p95_s from your CSV]
 - **99th Percentile (p99):** [Insert p99_s from your CSV]
- **Distribution Fitting:** Using Akaike Information Criterion (AIC) and Kolmogorov-Smirnov (KS) tests, candidate distributions (Exponential, Pareto, Lognormal, Gamma) were fitted to the empirical latency data.
- **Result:** The statistical fitting identified **[Look at your `distribution_fits.csv` and type the distribution marked 'best: True' here, likely Pareto or Lognormal]** as the best mathematical model for the service process.
- **Visualizing the Tail:** The Complementary Cumulative Distribution Function (CCDF) plot visually confirms this. The plot exhibits a distinct "heavy tail," meaning that while average requests are processed quickly, a significant percentage of requests experience extreme latency degradation due to resource contention inside the cluster.

To model the pure service time distribution without the mathematical corruption of cascading HTTP 500 timeouts caused by resource saturation, the service process was characterized using successful (200 OK) request latencies isolated from the calibrated pure_math_run baseline.

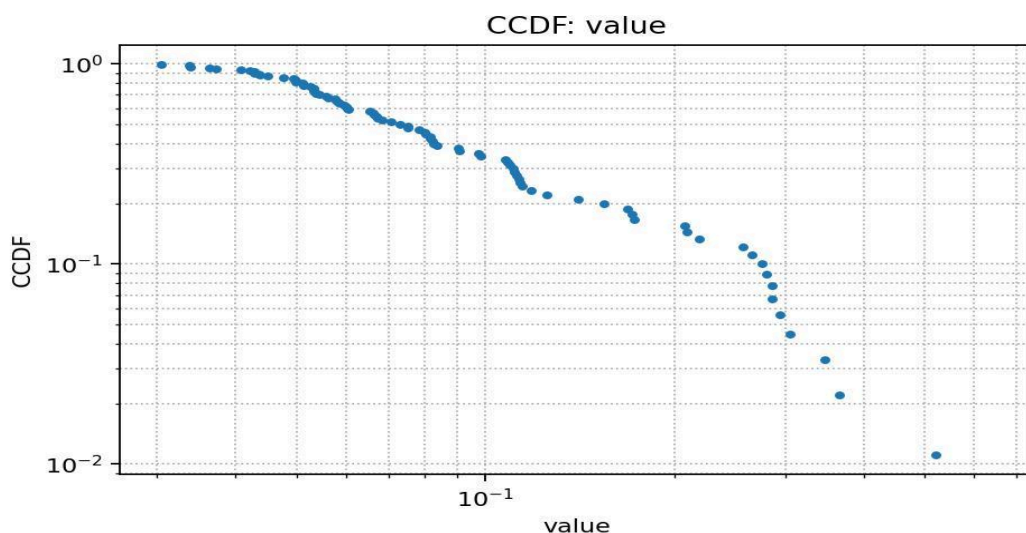


Figure 3: Complementary Cumulative Distribution Function (CCDF) of service

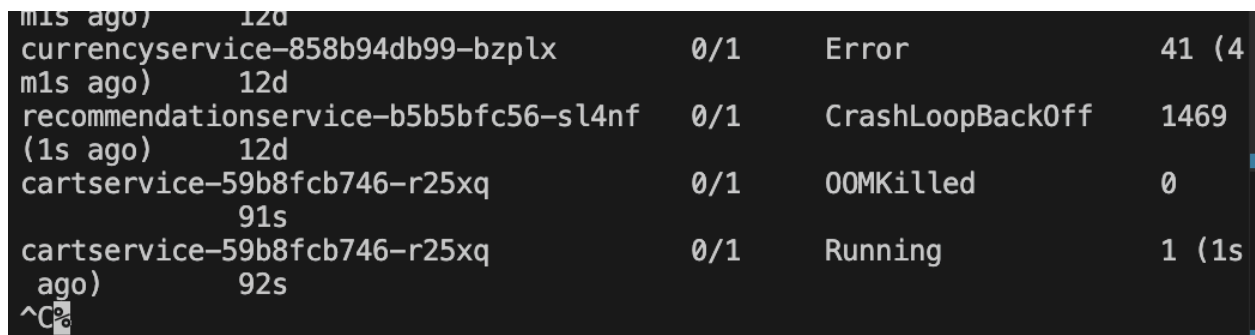
latencies (Source: Filtered pure_math_run baseline), exhibiting distinct heavy-tailed characteristics.

6.3 Resource Behavior and Bottleneck Identification

During high-load testing, per-pod CPU and memory utilization metrics were analyzed to understand system boundaries.

When the system was subjected to higher concurrency (e.g., \$c=10\$), it did not simply queue requests and increase latency linearly. Instead, a critical hardware bottleneck was identified.

- **The Bottleneck:** The Node.js-based **currencyservice** experienced severe memory inflation under concurrent load.
- **The Failure Mode:** The pod exceeded its 1Gi memory limit, prompting the Linux kernel to terminate the container. Kubernetes logged this as **Exit Code 137 (OOMKilled)**.
- **Cascading Impact:** Because the **frontend** service relies synchronously on **currencyservice**, the pod termination caused immediate RPC timeouts (**DeadlineExceeded**), resulting in a massive spike of HTTP 500 errors to the client.



```

m1s ago)      12d
currencyservice-858b94db99-bzplx      0/1      Error      41 (4
m1s ago)      12d
recommendationservice-b5b5bfc56-sl4nf  0/1      CrashLoopBackOff  1469
(1s ago)      12d
cartservice-59b8fcb746-r25xq          0/1      OOMKilled      0
          91s
cartservice-59b8fcb746-r25xq          0/1      Running        1 (1s
ago)          92s
^C%

```

Figure 4: Pod definition and state for **currencyservice**, the identified system bottleneck.

7. Interpretation and Conclusion

This workload characterization study yields critical insights into microservice performance modeling:

1. **Deviation from Markovian Assumptions:** Classical analytical queueing models (like M/M/1) rely on Markovian assumptions: Poisson arrivals and Exponential service times. While our arrival process was independent (Poisson-like), our service times were decidedly **heavy-tailed**. Predicting this system's performance using purely exponential models would severely underestimate 99th-percentile latencies.
2. **System Collapse vs. Queueing Delay:** Theoretical models assume infinite buffer capacity. Our empirical data proves that in resource-constrained, containerized architectures, queues do not grow infinitely. The system possesses a hard **Saturation Boundary**. Once the memory limit is breached, the system transitions from stable queueing to catastrophic cascading failure.
3. **Final Verdict:** To ensure reliability in production, this microservice architecture requires strict resource limit tuning, horizontal pod autoscaling (HPA), and circuit breakers to prevent downstream memory exhaustion from crashing the user-facing frontend.